

Intitulé du projet

Industrialisation, automatisation et sécurisation d'une application à travers une chaîne CI/CD

1. Contexte pédagogique

Dans le cadre du module **DevOps**, l'objectif est de se familiariser avec la culture DevOps et de mettre en œuvre les différents maillons de la chaîne d'intégration continue et de livraison continue. L'évaluation repose sur **100% la note du projet**.

Le projet sera réalisé en **mode Scrum**, par équipes de **3 à 4 étudiants**, sur une durée de **6 semaines de réalisation**, suivies d'une **7ème semaine de validation finale** sous forme de **présentation orale** et de **démonstration technique**.

Une **application existante** sera fournie aux équipes au démarrage, à **prendre en main un code source existant**, l'améliorer, l'industrialiser, l'automatiser, le superviser et le sécuriser dans une logique DevOps.

2. Acquis d'apprentissage

À travers ce projet, vous serez capables de :

- comprendre et appliquer les principes de la culture **DevOps** ;
- créer une **chaîne d'intégration continue avec Jenkins** ;
- créer et utiliser des **conteneurs Docker** pour le déploiement ;
- organiser les versions d'un projet avec **Git** ;
- construire un dépôt de livraison des artefacts avec **Nexus** ;
- tester les fonctionnalités implémentées avec **JUnit** ;
- interpréter les rapports de qualité de code avec **Sonar** ;
- construire un tableau de bord de supervision avec **Prometheus et Grafana** ;
- Sécuriser la chaîne CI/CD.

3. Objectif général du projet

Chaque équipe devra transformer l'application fournie en une solution **DevOps**, en mettant en place une chaîne complète et cohérente incluant :

- la gestion collaborative du code source ;
- l'intégration continue ;
- les tests automatisés ;
- l'analyse de la qualité du code ;
- la gestion des artefacts ;
- la conteneurisation et le déploiement ;

- la supervision continue ;
- la sécurisation de la chaîne CI/CD.

Cet objectif est directement cohérent avec les acquis d'apprentissage du module : découverte de la culture DevOps, CI avec Jenkins, conteneurisation avec Docker, gestion de versions avec Git, gestion des artefacts avec Nexus, tests avec JUnit, analyse Sonar, et supervision avec Prometheus/Grafana et enfin la sécurisation de la chaîne CI/CD.

4. Travail demandé

Chaque équipe devra travailler sur l'application fournie et produire une solution industrialisée respectant les exigences suivantes.

4.1. Prise en main et compréhension du projet existant

L'équipe devra :

- analyser l'architecture du projet fourni ;
- identifier les fonctionnalités déjà présentes ;
- repérer les limites techniques, organisationnelles ou qualitatives ;

4.2. Gestion du projet en mode Scrum

Le travail devra être organisé selon une logique **Scrum**, avec au minimum :

- une équipe de 3 à 4 étudiants ;
- une répartition des rôles ;
- un **Product Backlog** ;
- des **User Stories** ;
- un **Sprint Backlog** mis à jour chaque semaine ;
- un suivi régulier de l'avancement.

4.3. Gestion du code source

L'équipe devra :

- utiliser **Git** de manière structurée ;
- définir une stratégie simple de branches ;
- produire des commits explicites ;
- collaborer proprement sur le dépôt commun ;

4.4. Intégration continue

L'équipe devra mettre en place un pipeline **Jenkins** permettant au minimum :

- la récupération du code ;
- le build du projet ;
- l'exécution automatique des tests ;
- l'analyse de qualité ;

- la génération ou la publication des artefacts ;

4.5. Tests automatisés

L'équipe devra :

- identifier les fonctionnalités critiques ;
- implémenter des **tests unitaires** ;
- automatiser leur exécution dans la chaîne CI ;
- commenter la couverture et les limites des tests réalisés.

4.6. Vérification de la qualité du code

L'équipe devra intégrer **Sonar** afin de :

- détecter les problèmes de qualité ;
- interpréter les rapports ;
- corriger au moins les anomalies ou faiblesses prioritaires ;
- justifier les améliorations effectuées.

4.7. Gestion des artefacts

L'équipe devra utiliser **Nexus** pour :

- publier les builds ou artefacts générés ;
- versionner les livrables ;
- structurer un dépôt minimal exploitable.

4.8. Conteneurisation et déploiement

L'équipe devra :

- dockeriser l'application ;
- permettre son exécution dans un conteneur ;
- documenter clairement la procédure de lancement ;
- préparer l'application pour un environnement de test.

4.9. Supervision continue

L'équipe devra mettre en place un dispositif de supervision avec **Prometheus** et **Grafana** permettant d'observer au moins :

- la disponibilité de l'application ;
- quelques métriques système ou applicatives ;
- un tableau de bord synthétique ;
- quelques indicateurs utiles à l'anticipation des incidents.

5. Déroulement du projet sur 6 semaines

Semaine 1 : Gestion de versions avec Git et cadrage du projet

Objectifs

- prendre connaissance du projet et du code fourni ;
- installer et configurer l'environnement de travail ;
- initialiser l'organisation Scrum ;
- mettre en place le dépôt Git ;
- définir une stratégie de versioning et de collaboration ;
- établir le backlog initial et les premières user stories.

Livrables attendus

- composition de l'équipe ;
- répartition initiale des rôles ;
- backlog initial ;
- document court de compréhension du projet ;
- dépôt Git initialisé et structuré ;
- stratégie de branches définie ;
- premières tâches planifiées dans le tableau Scrum.

Semaine 2 : Intégration continue avec Jenkins

Objectifs

- mettre en place le serveur Jenkins ;
- créer un pipeline d'intégration continue ;
- automatiser les premières étapes de build ;
- intégrer l'exécution automatique du projet ;
- relier Jenkins au dépôt Git ;
- structurer le pipeline dans un Jenkinsfile.

Livrables attendus

- pipeline Jenkins fonctionnel ;
- Jenkinsfile documenté ;
- preuve de connexion entre Git et Jenkins ;
- exécution automatique d'un build ;
- tableau d'avancement Scrum mis à jour.

Semaine 3 : Qualité du code avec Sonar et gestion des artefacts avec Nexus

Objectifs

- intégrer Sonar dans la chaîne CI ;
- analyser la qualité du code source ;
- interpréter les résultats d'analyse ;
- corriger les premiers problèmes critiques détectés ;
- mettre en place Nexus ;
- publier les premiers artéfacts du projet ;
- structurer la gestion des versions des livrables.

Livrables attendus

- analyse Sonar intégrée au pipeline ;
- rapport Sonar commenté ;
- premières corrections de qualité réalisées ;
- dépôt Nexus configuré ;
- premier artéfact publié dans Nexus ;
- démonstration du pipeline enrichi.

Semaine 4 : Conteneurisation et préparation du déploiement avec Docker

Objectifs

- dockeriser l'application ;
- créer les fichiers nécessaires au déploiement conteneurisé ;
- préparer l'exécution du projet dans un environnement isolé ;
- améliorer le pipeline Jenkins pour intégrer la phase Docker ;
- tester le lancement de l'application dans un conteneur.

Livrables attendus

- Dockerfile et/ou docker-compose ;
- application exécutable dans un conteneur ;
- documentation de lancement ;
- intégration de l'étape Docker dans la chaîne CI/CD ;
- démonstration du déploiement conteneurisé.

Semaine 5 : Supervision continue avec Prometheus et Grafana

Objectifs

- mettre en place un système de supervision continue ;
- configurer Prometheus pour collecter des métriques ;
- configurer Grafana pour visualiser les données ;
- construire un tableau de bord permettant de suivre le comportement de l'application ou de l'infrastructure ;
- identifier les indicateurs pertinents pour anticiper les incidents.

Livrables attendus

- Prometheus configuré ;
- Grafana configuré ;
- tableau de bord opérationnel ;
- métriques collectées et exploitées ;
- démonstration de la supervision continue.

Semaine 6 : Sécurisation de la chaîne CI/CD

Objectifs

La sixième semaine est entièrement consacrée à la **sécurisation du projet et de la chaîne CI/CD**.

L'équipe devra :

- sécuriser la gestion des identifiants, mots de passe et secrets ;
- éviter les mauvaises pratiques dans le dépôt Git ;
- analyser les dépendances utilisées et vérifier les vulnérabilités éventuelles ;
- renforcer la sécurité des images et de la configuration Docker ;
- examiner les risques de sécurité applicative ;
- intégrer une analyse basée sur les principes de l'**OWASP Top 10** ;
- proposer des mesures correctives réalistes.

Livrables attendus

Chaque équipe devra être capable de commenter au moins :

- la gestion des entrées utilisateur ;
- la gestion des erreurs ;
- l'exposition des données sensibles ;
- la sécurité des dépendances ;
- la sécurité du pipeline Jenkins ;
- la sécurité de l'image Docker ;
- les défauts de configuration possibles ;
- les risques applicatifs les plus pertinents au regard de l'**OWASP Top 10**.

Livrable spécifique semaine 6

- rapport de sécurisation de la chaîne CI/CD, incluant :
 - les risques identifiés ;
 - les mesures mises en place ;
 - les vulnérabilités restantes ;
 - les recommandations d'amélioration.

Semaine 7 : Validation finale

La 7ème semaine sera dédiée à la validation du projet.

Chaque équipe devra réaliser :

Une présentation orale

La présentation devra exposer :

- le contexte et la compréhension du projet ;
- l'organisation Scrum adoptée ;
- les choix techniques ;
- l'architecture mise en place ;
- la gestion du code source avec Git ;
- le pipeline Jenkins ;
- l'analyse Sonar ;
- la publication des artefacts dans Nexus ;
- la conteneurisation avec Docker ;
- la supervision avec Prometheus et Grafana ;
- les mesures de sécurité mises en place.

Une démonstration technique

La démonstration devra montrer concrètement :

- le dépôt Git ;
- le fonctionnement du pipeline Jenkins ;
- le rapport Sonar ;
- la publication dans Nexus ;
- le lancement de l'application dans Docker ;
- le tableau de bord Grafana ;
- les métriques collectées via Prometheus ;
- les éléments de sécurisation intégrés dans la chaîne CI/CD.

5. Livrables finaux attendus

À la fin du projet, chaque équipe devra remettre :

- le lien du dépôt Git ;
- le backlog et le suivi Scrum ;
- le Jenkinsfile ou le script pipeline ;
- les fichiers Docker ;
- la documentation d'installation et d'exécution ;
- les tests automatisés ;
- le rapport Sonar commenté ;
- la preuve de publication dans Nexus ;
- les captures ou exports du monitoring ;
- le rapport de sécurité de la semaine 6 ;
- le support de présentation finale.

6. Critères d'évaluation

L'évaluation du projet sera divisée comme suit :

- **Organisation Scrum et gestion d'équipe : 15%**
- **Gestion de versions avec Git : 10%**
- **Pipeline Jenkins / intégration continue : 15%**
- **Tests automatisés : 10%**
- **Qualité du code / Sonar : 10%**
- **Gestion des artefacts avec Nexus : 10%**
- **Docker / déploiement : 10%**
- **Monitoring avec Prometheus/Grafana : 10%**
- **Sécurisation CI/CD / OWASP Top 10 : 10%**